Volatility Forecasting and Risk Management Using GARCH Models

Fenglei Wu

1) Introduction

Financial markets exhibit time-varying volatility, characterized by periods of relative calm followed by sudden spikes in uncertainty. Accurately forecasting this volatility is critical for risk management, portfolio allocation, and regulatory capital planning. However, a key challenge in practice is that volatility dynamics are not only persistent (volatility clustering), but also often asymmetric, with negative shocks having a larger impact than positive ones. Traditional constant-variance models fail to capture these features, motivating the use of Generalized Autoregressive Conditional Heteroskedasticity (GARCH) family models.

This project investigates the performance of several GARCH-type models in forecasting volatility and managing risk for major U.S. equity ETFs. Specifically, it compares the standard GARCH(1,1) model with two asymmetric extensions, Exponential GARCH (EGARCH) and GJR-GARCH, using daily return data for three widely traded exchange-traded funds (SPY, QQQ, and DIA). Out-of-sample forecasting performance is evaluated using standard loss functions, and the best-performing model is subsequently applied to Value-at-Risk (VaR) estimation and backtesting.

The objective of this analysis is twofold. First, it evaluates whether incorporating asymmetric volatility dynamics improves predictive accuracy relative to the symmetric GARCH benchmark. Second, it examines whether improved volatility forecasts translate into more reliable risk estimates by assessing VaR performance through statistical backtesting procedures. By linking model performance to practical risk applications, this study provides insights into the trade-off between model complexity and real-world forecasting effectiveness.

2) Data and stylized facts

2.1) Data Description

The analysis uses daily closing price data for three major U.S. equity ETFs: SPY (S&P 500), QQQ (Nasdaq-100), and DIA (Dow Jones Industrial Average). The sample spans from

January 1, 2015, to January 1, 2025, covering a range of market conditions, including periods of stability as well as extreme volatility during the COVID-19 crisis. Data is obtained from Yahoo Finance via the yfinance Python library.

Daily log returns are computed from adjusted closing prices. In addition, a rolling window variance is used as a proxy for realized volatility in the forecast evaluation stage, serving as a benchmark against model-based forecasts.

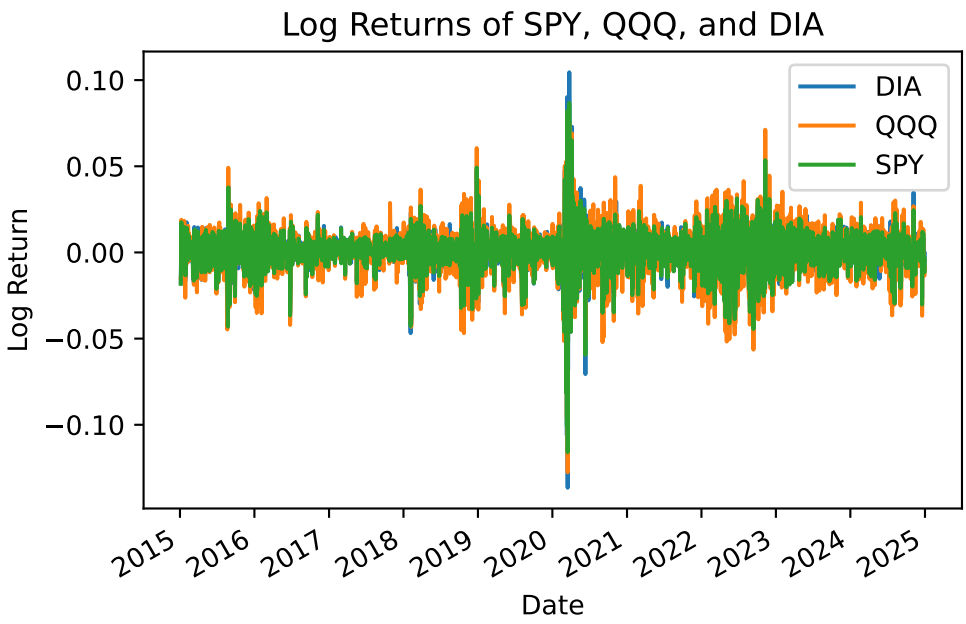
2.2) Returns and volatility characteristics

The return series exhibit several well-known stylized facts in financial time series. First, log returns fluctuate around zero with no clear trend, indicating weak predictability in the mean. However, the magnitude of returns varies over time, revealing strong volatility clustering, where large movements (both positive and negative) tend to be followed by further large movements.

Second, squared returns highlight this clustering behavior more clearly, showing persistent periods of high and low volatility. This suggests the presence of conditional heteroskedasticity, which violates the constant variance assumption of classical time series models.

These empirical observations provide strong motivation for using GARCH-type models, which explicitly model time-varying conditional variance and are designed to capture volatility persistence observed in financial markets.

```
Ticker           DIA       QQQ       SPY
Date
2015-01-05 -0.017576 -0.014777 -0.018225
2015-01-06 -0.008328 -0.013499 -0.009464
2015-01-07  0.012608  0.012809  0.012385
2015-01-08  0.017893  0.018959  0.017589
2015-01-09 -0.008652 -0.006605 -0.008046
```


Log Returns of SPY, QQQ, and DIA
DIA
QQQ
SPY
Log Return
0.10
0.05
0.00
−0.05
−0.10
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
Date

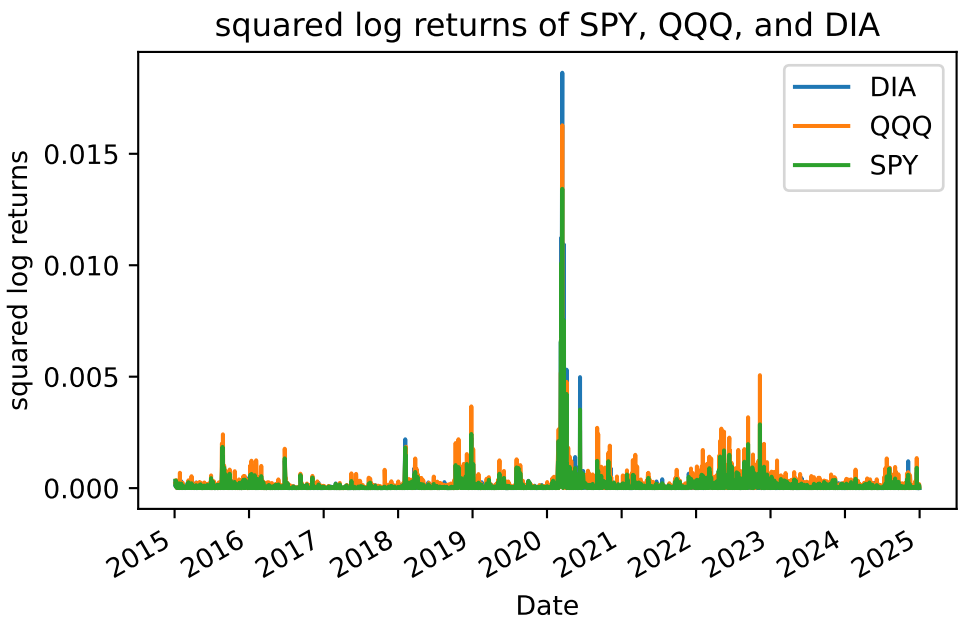

squared log returns of SPY, QQQ, and DIA
DIA
QQQ
SPY
squared log returns
0.015
0.010
0.005
0.000
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
Date

Figure 1: Daily Log Returns of SPY, QQQ, and DIA (2015-2025)

Figure 1 displays the daily log returns of SPY, QQQ, and DIA from 2015 to 2025. The log returns fluctuate around zero with no clear serial correlation in the mean. However, periods of large absolute returns tend to cluster together, suggesting time-varying volatility.

Figure 2: Squared Log Returns of SPY, QQQ, and DIA (2015-2025)

Figure 2 shows the squared log returns of SPY, QQQ, and DIA over the same period. Squared returns provide a clearer representation of volatility dynamics. The persistence of large values over time indicates strong volatility clustering, which is a key stylized fact in financial time series and suggests the presence of conditional heteroskedasticity.

2.3) Summary statistics and ACF analysis

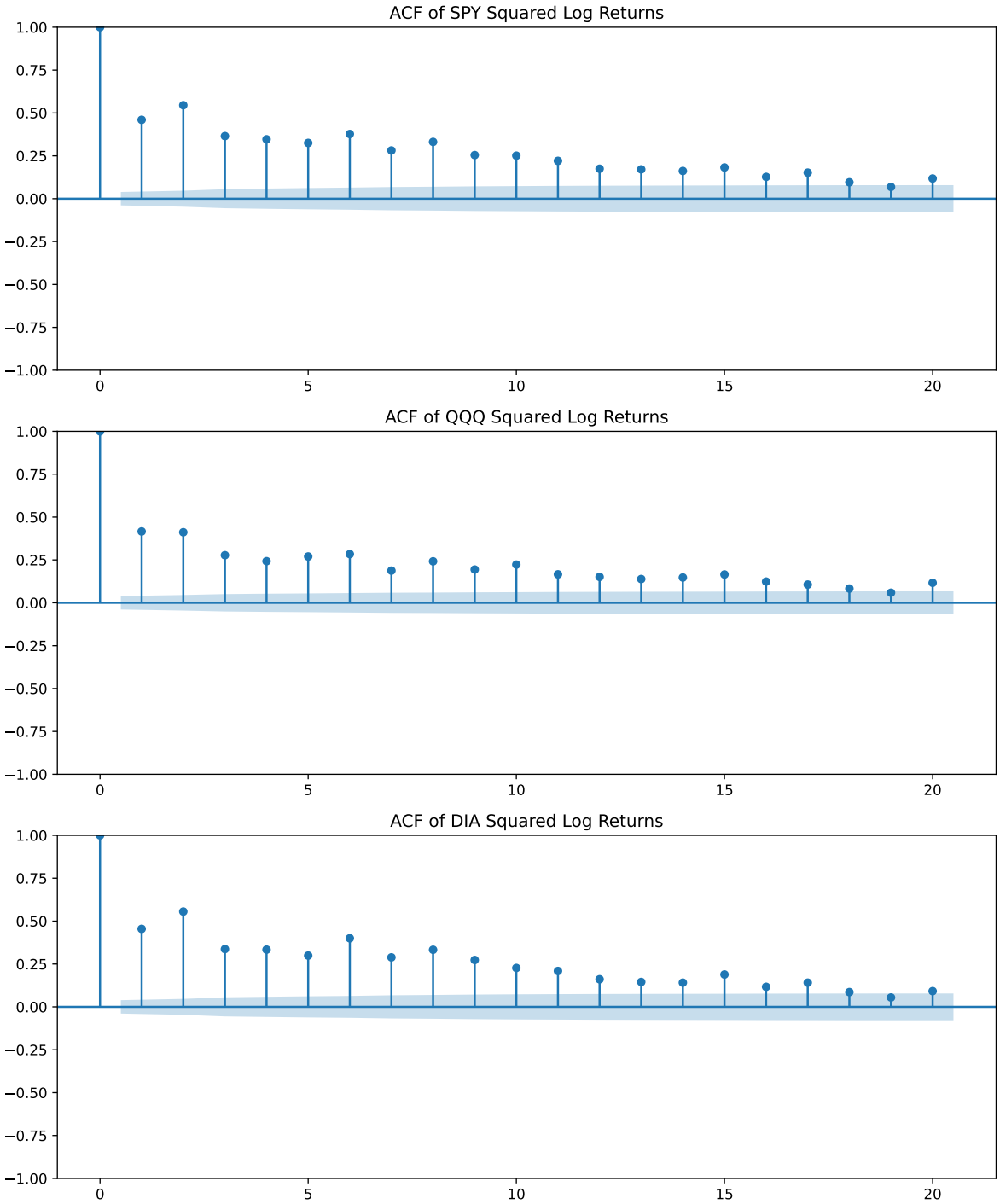
The summary statistics reveal key non-normal features of financial returns. Negative skewness indicates a higher likelihood of extreme negative returns (downside risk), while excess kurtosis suggests heavy tails and a higher probability of extreme market movements.

```
Log Returns Statistics
            Mean  standard deviation  skewness   kurtosis       min       max
Ticker
DIA     0.000429            0.011043 -0.983438  22.297463 -0.136520  0.104533
QQQ     0.000668            0.013781 -0.553143   6.849144 -0.127592  0.081309
SPY     0.000486            0.011137 -0.800717  13.663904 -0.115886  0.086731
Square log Returns Statistics
            Mean  standard deviation   skewness    kurtosis  min       max
Ticker
DIA     0.000122            0.000599  19.169885  476.970757  0.0  0.018638
QQQ     0.000190            0.000561  13.929024  316.239196  0.0  0.016280
SPY     0.000124            0.000488  15.884707  339.766317  0.0  0.013430
```

Log Returns Statistics

Conclusion 1: heavier left tail (downside risk) The return distributions exhibit negative skewness, indicating that large negative returns occur more frequently than large positive returns. Conclusion 2: heavy tails / extreme events The return series exhibit excess kurtosis, suggesting heavy tails and a higher probability of extreme market movements. ### Square Log Returns Statistics Conclusion 3: The statistics of squared returns further confirm the presence

of volatility clustering, as indicated by high persistence and extreme values during turbulent periods.



ACF Log return

The ACF of log returns shows little to no significant autocorrelation, indicating that returns are approximately serially uncorrelated.

ACF Squared Log return (Most important)

The ACF of squared returns shows significant positive autocorrelation, indicating volatility clustering and persistence in conditional variance.This suggests that periods of high volatility tend to be followed by high volatility, which motivates the use of GARCH-family models.

This pattern suggests that while returns are approximately uncorrelated, their variance is highly persistent over time, indicating conditional heteroskedasticity. This provides strong justification for using GARCH-type models to capture time-varying volatility.

3) Volatility modeling with GARCH family models

3.1)ARCH Test and Evidence of Conditional Heteroskedasticity

```
===== ARCH Test =====

----DIA----
LM Statistic 1027.580963245416
P-value 0.0
F Statistic 173.46378099768415
F_P-value 0.0
Good for GARCH modeling

----QQQ----
LM Statistic 697.8664424563517
P-value 0.0
F Statistic 96.31158141138681
F_P-value 0.0
Good for GARCH modeling

----SPY----
LM Statistic 962.3074202679906
P-value 0.0
F Statistic 155.57180592424228
F_P-value 0.0
Good for GARCH modeling
```

Before estimating GARCH-type models, it is necessary to formally test whether the return series exhibit conditional heteroskedasticity. The ARCH-LM test is applied to each ETF return series to detect the presence of time-varying variance.

The results indicate that all three ETFs (SPY, QQQ, and DIA) exhibit statistically significant ARCH effects, with p-values effectively close to zero. This leads to a strong rejection of the null hypothesis of homoskedasticity.

These findings confirm that the variance of returns is not constant over time but instead depends on past information, indicating the presence of conditional heteroskedasticity. This result is consistent with the earlier ACF analysis of squared returns, which showed persistent autocorrelation.

Therefore, the ARCH test provides formal statistical evidence supporting the use of GARCH-family models, which are specifically designed to model time-varying conditional volatility.

3.2) Model Estimation and Comparison

To examine volatility dynamics, three GARCH-family models are estimated for each ETF: GARCH(1,1), EGARCH, and GJR-GARCH. Estimation is conducted via maximum likelihood on rescaled daily log returns (multiplied by 100) to ensure numerical stability without altering the underlying volatility structure.

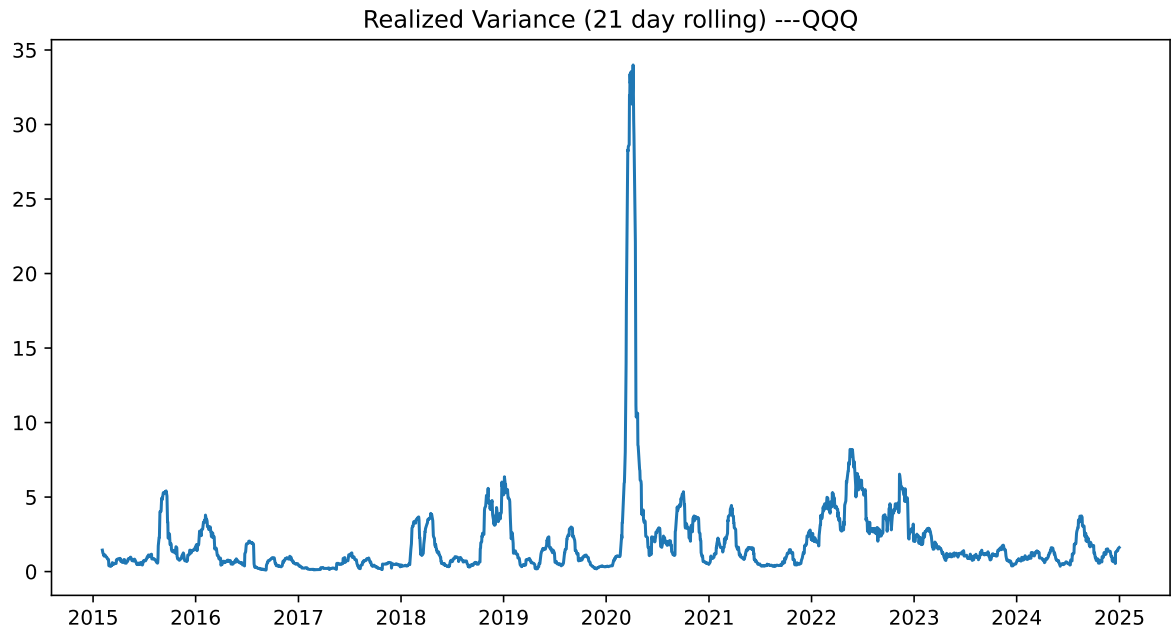
Asset	Model	mu	omega	alpha	beta	gamma	alpha+beta	AIC
DIA	GARCH(1,1)	0.0782	0.0397	0.1838	0.7797	-	0.9635	6243.81
DIA	EGARCH	0.0464	-0.0077	0.2424	0.9521	-0.1349	-	6171.84
DIA	GJR-GARCH	0.0513	0.0364	0.0556	0.8015	0.2128	-	6180.49
QQQ	GARCH(1,1)	0.1031	0.0495	0.1372	0.8378	-	0.975	7886.92
QQQ	EGARCH	0.0530	0.0213	0.1969	0.9571	-0.1226	-	7819.38
QQQ	GJR-GARCH	0.0644	0.0487	0.0345	0.8585	0.1521	-	7838.18
SPY	GARCH(1,1)	0.0867	0.0401	0.1888	0.7801	-	0.9689	6448.11
SPY	EGARCH	0.0490	-0.0058	0.2552	0.9471	-0.1598	-	6359.86
SPY	GJR-GARCH	0.0538	0.0391	0.0607	0.7940	0.2246	-	6386.43

Table X reports the key parameter estimates for the GARCH-family models across the three ETF return series. The GARCH(1,1) results show that the sum of ($\alpha + \beta$) is close to one for all assets, indicating strong volatility persistence. In the asymmetric models, the (γ) parameter captures leverage effects: (γ) is negative in EGARCH and positive in GJR-GARCH, both of which imply that negative shocks have a stronger impact on volatility than positive shocks. In addition, EGARCH achieves the lowest AIC for all three ETFs, suggesting the best in-sample fit among the competing specifications. However,i mproved in-sample fit does not necessarily imply better out-of-sample forecasting performance, which is examined in the next section.

4) Forecast evaluation

4.1) Rolling Variance as Proxy for Realized Volatility

```
Ticker            DIA       QQQ       SPY
Date
2024-12-24  0.624013  1.428230  0.749815
2024-12-26  0.566515  1.432287  0.745882
2024-12-27  0.572584  1.540202  0.789683
2024-12-30  0.602916  1.605009  0.846804
2024-12-31  0.575340  1.613966  0.825165
```



For illustration, A 21-day rolling variance is used as a proxy for realized volatility and serves as the benchmark for forecast evaluation. This rolling-window measure captures local return variability and provides a practical target against which model-based conditional variance forecasts can be compared. For illustration, QQQ is shown as a representative asset, while the same procedure is applied to SPY and DIA in the subsequent forecast evaluation.

4.2) Conduct out-of-sample forecast evaulation using loss functions

To evaluate out-of-sample forecasting performance, the sample is split into a training period ending on December 31, 2022 and a test period beginning on January 1, 2023. An expanding-window forecasting scheme is used, where each model is recursively re-estimated as new ob-

servations become available. One-step-ahead variance forecasts are then compared against the rolling variance benchmark using MSE, MAE, and QLIKE loss functions.

```
===== Forecast Evaluation Results for All Assets =====
  Asset       Model       MSE       MAE     QLIKE
0   QQQ  GARCH(1,1)  0.188578  0.315464  0.039580
1   QQQ      EGARCH  0.305093  0.378246  0.055109
2   QQQ   GJR-GARCH  0.298297  0.377440  0.054553
3   SPY  GARCH(1,1)  0.098131  0.218566  0.070167
4   SPY      EGARCH  0.178302  0.283199  0.102478
5   SPY   GJR-GARCH  0.188725  0.277661  0.102095
6   DIA  GARCH(1,1)  0.065122  0.182815  0.069007
7   DIA      EGARCH  0.096038  0.228126  0.092881
8   DIA   GJR-GARCH  0.101293  0.217479  0.092045
```

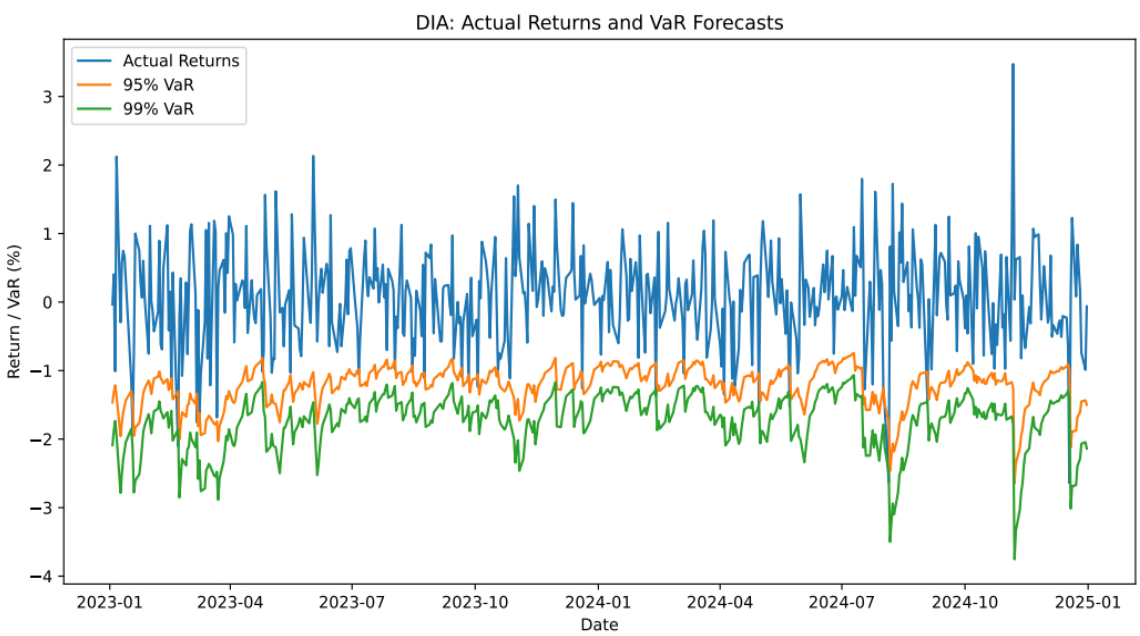
The out-of-sample forecast evaluation shows that the standard GARCH(1,1) model delivers the best volatility forecasts across all three ETFs, achieving the lowest MSE, MAE, and QLIKE values relative to EGARCH and GJR-GARCH. Although the asymmetric models capture leverage effects in-sample, this added flexibility does not translate into superior predictive performance out of sample. This suggests that, for broad equity ETFs at the daily frequency, volatility persistence is the dominant feature, while asymmetry is not sufficiently strong to justify the additional model complexity. In this dataset, the simpler GARCH(1,1) specification appears to be the most robust forecasting model.

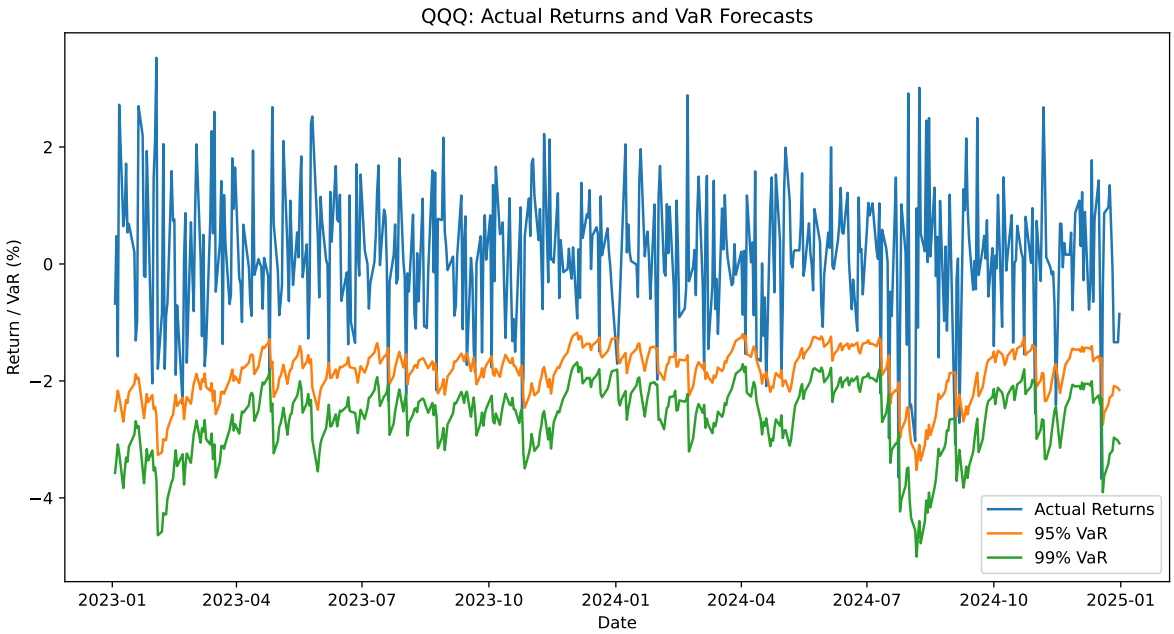
5) VaR forecasting and backtesting

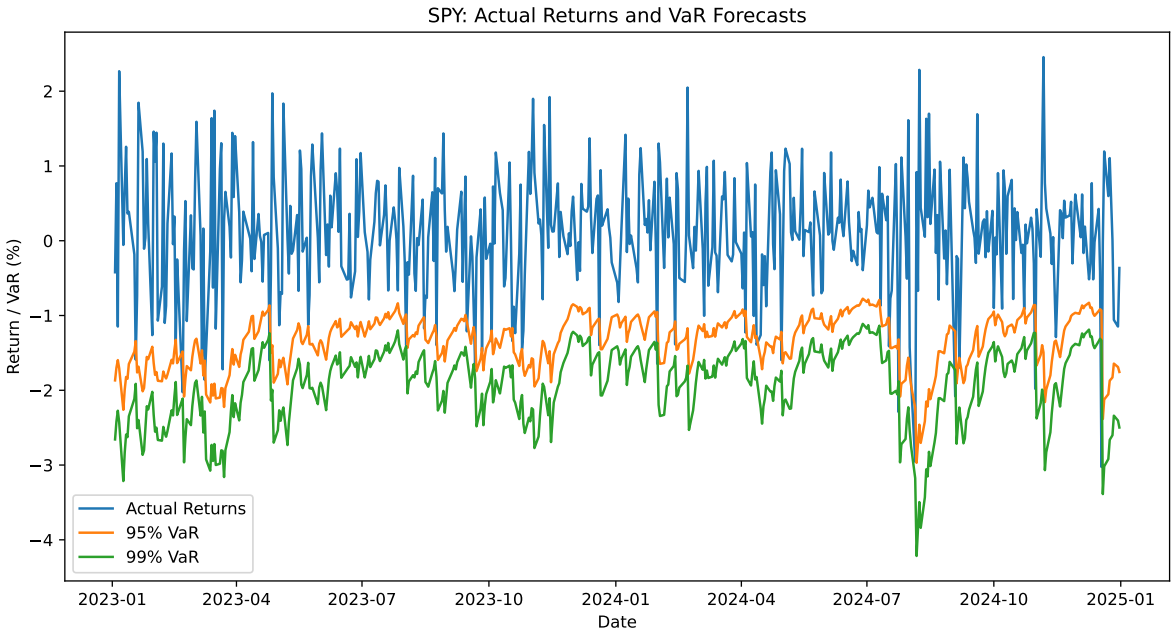
Apply to VaR forecasting and risk managemnt applications

To evaluate the practical usefulness of volatility forecasts in risk management, Value-at-Risk (VaR) forecasts are constructed using the GARCH(1,1) model. The same expanding-window framework used in volatility forecasting is applied to generate one-step-ahead variance forecasts, which are then converted into volatility and VaR estimates. The performance of the VaR forecasts is evaluated using violation rates and the Kupiec unconditional coverage test.

```
===== VaR Backtesting Summary for All Assets =====
Asset VaR Level  Expected Violation Rate  Actual Violation Rate  Number of Violations  Total Observations
  QQQ       95%                     0.05               0.053785                    27                 502
  QQQ       99%                     0.01               0.017928                     9                 502
  SPY       95%                     0.05               0.055777                    28                 502
  SPY       99%                     0.01               0.011952                     6                 502
  DIA       95%                     0.05               0.047809                    24                 502
  DIA       99%                     0.01               0.011952                     6                 502


  ===== Kupiec Backtest Results =====

  --- QQQ ---
  Asset VaR Level  Kupiec LR Statistic  Kupiec p-value
    QQQ       95%             0.147907        0.700544
    QQQ       99%             2.580262        0.108204

  --- SPY ---
  Asset VaR Level  Kupiec LR Statistic  Kupiec p-value
    SPY       95%             0.340524        0.559527
    SPY       99%             0.181888        0.669756

  --- DIA ---
  Asset VaR Level  Kupiec LR Statistic  Kupiec p-value
    DIA       95%             0.051463        0.820538
    DIA       99%             0.181888        0.669756
```




QQQ: Actual Returns and VaR Forecasts
Return / VaR (%)
2
0
−2
−4
Actual Returns
95% VaR
99% VaR
2023-01
2023-04
2023-07
2023-10
2024-01
2024-04
2024-07
2024-10
2025-01
Date


SPY: Actual Returns and VaR Forecasts
Return / VaR (%)
2
1
0
−1
−2
−3
−4
Actual Returns
95% VaR
99% VaR
2023-01
2023-04
2023-07
2023-10
2024-01
2024-04
2024-07
2024-10
2025-01
Date

The VaR backtesting results show that the actual violation rates are close to the expected levels at both the 95% and 99% confidence levels across all three ETFs. In addition, the Kupiec test fails to reject the null hypothesis of correct coverage, as the p-values are sufficiently large.

These results indicate that the GARCH(1,1)-based VaR forecasts provide an accurate representation of downside risk and are statistically consistent with the expected violation frequencies. Therefore, the volatility forecasts generated by the GARCH model can be effectively translated into reliable risk measures for practical risk management applications.

This finding highlights that while more complex models do not improve volatility forecasting performance, accurate volatility estimation itself remains sufficient for reliable risk management.

6) Risk management discusssion

The volatility and VaR forecasts derived from the GARCH(1,1) model provide a practical, forward-looking measure of market risk. Because the model produces time-varying conditional variance, it enables dynamic risk management actions such as position scaling and capital allocation. When predicted volatility increases, the corresponding VaR thresholds become more conservative (i.e., more negative), signaling higher potential downside risk and justifying reduced exposure or increased capital buffers.

Backtesting results indicate that the empirical violation rates are close to the nominal levels at both the 95% and 99% VaR thresholds, and the Kupiec test does not reject correct coverage. This suggests that the GARCH-based VaR forecasts are well-calibrated and can adapt to changing volatility regimes, particularly in the presence of volatility clustering.

Importantly, these findings highlight that a relatively simple symmetric model can deliver reliable risk estimates in practice. Despite the presence of asymmetric effects in-sample, the addedc omplexity of EGARCH and GJR-GARCH does not translate into improved forecasting performance, while GARCH(1,1) remains robust for risk management applications. However, VaR should not be used as a standalone risk measure. It does not capture the severity of losses beyond the VaR threshold and may underestimate tail risk during extreme market conditions. Therefore, it should be complemented with additional measures such as Expected Shortfall (ES) to provide a more comprehensive assessment of downside risk.

7) Limitations

While the proposed modeling framework produces reasonable volatility and VaR forecasts, several limitations should be acknowledged.

First, the VaR computation relies on the assumption of conditional normality. Financial returns are well known to exhibit fat tails and skewness, which may lead to systematic underestimation of extreme downside risk.

Second, VaR provides only a quantile-based threshold and does not capture the magnitude of losses beyond that threshold. As a result, it fails to measure tail risk severity. Complementary measures such as Expected Shortfall (ES) would provide a more comprehensive assessment of downside risk.

Third, the GARCH-family models assume stable volatility dynamics over time. In the presence of structural breaks or regime shifts, particularly during financial crises, model performance may deteriorate and risk forecasts may become less reliable.

Fourth, the forecast evaluation relies on a rolling variance proxy rather than high-frequency realized volatility. While practical, this approximation may introduce measurement error and affect the accuracy of model comparison.

Finally, parameter estimates are assumed to be stable over the sample period. In reality, model parameters may vary over time, and adaptive or rolling estimation techniques could improve robustness.

Overall, these limitations suggest that while the model provides useful short-term volatility and risk forecasts, its outputs should be interpreted with caution, especially during periods of market stress.

8) Conclusion

This study evaluates volatility forecasting and risk measurement using GARCH-family models applied to major equity ETFs. The empirical results lead to two main findings.

First, although asymmetric models such as EGARCH and GJR-GARCH capture leverage effects in-sample, they do not improve out-of-sample forecasting performance. The standard GARCH(1,1) model consistently delivers the lowest MSE, MAE, and QLIKE across QQQ, SPY, and DIA, indicating that volatility persistence is the dominant feature of daily equity returns, while asymmetry does not provide additional predictive power in this context.

Second, volatility forecasts from the GARCH(1,1) model translate effectively into risk measures. VaR estimates at the 95% and 99% levels are well calibrated, with empirical violation rates close to theoretical expectations and Kupiec tests failing to reject correct coverage. This suggests that GARCH-based volatility forecasts can be reliably used for practical risk management.

Taken together, the results highlight an important implication: increased model complexity does not necessarily lead to better forecasting or risk estimation performance. In this setting, a simple and robust specification such as GARCH(1,1) is sufficient to capture time-varying market risk and support risk management decisions.

Nevertheless, the analysis is subject to several limitations, including distributional assumptions and the lack of explicit tail-risk measures. Future work may extend the framework by incorporating heavy-tailed distributions or alternative risk metrics such as Expected Shortfall to improve robustness under extreme market conditions.

Appendix A: Data processing Code:

```
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

#Download historical data
tickers = ["SPY","QQQ","DIA"] ## They are major ETFs that track the S&P 500, Nasdaq-100, and
data = yf.download(tickers, start="2015-01-01", end="2025-01-01")["Close"]  #10 years of data
#Calculate log returns
log_returns = np.log(data/data.shift(1)).dropna()

print(log_returns.head())

#Plot the log returns
```

```
plt.figure(figsize=(12,6))
log_returns.plot()
plt.title("Log Returns of SPY, QQQ, and DIA")
plt.xlabel("Date")
plt.ylabel("Log Return")
plt.legend()
plt.show()

#Plot squared log returns
plt.figure(figsize=(12,6))
(log_returns**2).plot()
plt.title("squared log returns of SPY, QQQ, and DIA")
plt.xlabel("Date")
plt.ylabel("squared log returns")
plt.legend()
plt.show()
```

Appendix B: Summary statistics and ACF analysis code

```
import pandas as pd
## Calculate and print summary statistics for log returns and squared log returns
print("Log Returns Statistics")  ## Log returns contain both positive and negative values, so
summary_log_returns = pd.DataFrame({
    "Mean" : log_returns.mean(),
    "standard deviation" : log_returns.std(),
    "skewness": log_returns.skew(),
    "kurtosis":log_returns.kurt(),
    "min":log_returns.min(),
    "max":log_returns.max(),
})
print(summary_log_returns)

print("Square log Returns Statistics")
square_log_returns = log_returns**2
summary_square_log_returns = pd.DataFrame({
    "Mean" : square_log_returns.mean(),
    "standard deviation" : square_log_returns.std(),
    "skewness": square_log_returns.skew(),
    "kurtosis":square_log_returns.kurt(), ## Value is bigger = more extreme events
    "min":square_log_returns.min(),
```

```
    "max":square_log_returns.max(),
})

# Calculate the ACF for log returns
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

# ACF of log returns
fig, axes = plt.subplots(3, 1, figsize=(10, 12))

for i, ticker in enumerate(tickers):
    plot_acf(log_returns[ticker]**2, lags=20, ax=axes[i])
    axes[i].set_title(f"ACF of {ticker} Squared Log Returns")

plt.tight_layout()
plt.show()

# ACF of squared log returns
fig, axes = plt.subplots(3, 1, figsize=(10, 12))

for i, ticker in enumerate(tickers):
    plot_acf(log_returns[ticker], lags=20, ax=axes[i])
    axes[i].set_title(f"ACF of {ticker} Log Returns")

plt.tight_layout()
plt.show()
```

Appendix C: GARCH model estimation code

```
## GARCH(1,1) = Models volatility persistence, but does not distinguish good vs bad news
## EGARCH = Allows asymmetry and models log variance to ensure positivity
## GJR-GARCH = Allows negative shocks to have a larger effect on volatility than positive sho
# Why does your project require all three?
# because the goal is not to fit just any one model, but to compare different volatility mode

## Apply arch test before we run GARCH model
# Arch test is for validating whether GARCH modeling is appriate
from statsmodels.stats.diagnostic import het_arch
print("===== ARCH Test =====")
```

```python
for col in log_returns.columns:  ## Log_returns.columns = {QQQ,DIA,SPY}
    print(f"\n----{col}----")
    arch_test = het_arch(log_returns[col].dropna(), nlags = 10)

    lm_stat = arch_test[0]
    lm_pvalue = arch_test[1]
    F_stat = arch_test[2]
    F_pvalue = arch_test[3]

    print("LM Statistic",lm_stat)
    print("P-value",round(lm_pvalue,8))
    print("F Statistic",F_stat)
    print("F_P-value",round(F_pvalue,8))

    if lm_pvalue < 0.05:
        print("Good for GARCH modeling")
    else:
        print("No significant ARCH effect detected: GARCH may not be appropriate")


from arch import arch_model
import pandas as pd
import numpy as np

# Rescale returns
log_returns_Rescaling = log_returns * 100

# Model specifications
model_specs = {
    "GARCH(1,1)": {"vol": "GARCH", "p": 1, "q": 1, "o": 0},
    "EGARCH": {"vol": "EGARCH", "p": 1, "q": 1, "o": 1},
    "GJR-GARCH": {"vol": "GARCH", "p": 1, "q": 1, "o": 1},
}

# Store results
rows = []

for asset in tickers:
    for model_name, spec in model_specs.items():
        model = arch_model(
            log_returns_Rescaling[asset].dropna(),
            vol=spec["vol"],
```

```
            p=spec["p"],
            q=spec["q"],
            o=spec["o"],
            mean="constant"
        )

        res = model.fit(disp="off")
        params = res.params

        rows.append({
            "Asset": asset,
            "Model": model_name,
            "mu": params.get("mu", np.nan),
            "omega": params.get("omega", np.nan),
            "alpha[1]": params.get("alpha[1]", np.nan),
            "beta[1]": params.get("beta[1]", np.nan),
            "gamma[1]": params.get("gamma[1]", np.nan),
            "AIC": res.aic
        })

model_summary_df = pd.DataFrame(rows)

# Make the table cleaner
model_summary_df = model_summary_df.round(4)

print(model_summary_df)
```

Appendix D: Forecast evaluation code

```
realized_var = log_returns_Rescaling.rolling(window = 21).var() ## 21 trading day in a month
realized_var = realized_var.dropna()
print(realized_var.tail())  ## Last defalut=5 rows data

import matplotlib.pyplot as plt

asset = "QQQ"
plt.figure(figsize =(10,5))
plt.plot(realized_var[asset])
plt.title("Realized Variance (21 day rolling) ---QQQ")
plt.show()
```

```python
from arch import arch_model
import numpy as np
import pandas as pd
# Define training and testing periods
train_end = "2022-12-31"
test_start = "2023-01-01"

train_returns = log_returns_Rescaling.loc[:train_end]
test_returns = log_returns_Rescaling.loc[test_start:]

train_variance = realized_var.loc[:train_end]
test_variance = realized_var.loc[test_start:]

# Define model specifications for GARCH(1,1), EGARCH, and GJR-GARCH
model_specs = {
    "GARCH(1,1)": {"vol": "GARCH", "p": 1, "q": 1, "o": 0},
    "EGARCH": {"vol": "EGARCH", "p": 1, "q": 1, "o": 1},
    "GJR-GARCH": {"vol": "GARCH", "p": 1, "q": 1, "o": 1},
}

# Define loss functions for forecast evaluation
def mse_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def mae_loss(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

def qlike_loss(y_true, y_pred):
    return np.mean(y_true / y_pred - np.log(y_true / y_pred) - 1)

# Define a function to evaluate forecasts for a given asset and model specifications
def evaluate_asset_forecast(asset, train_returns, test_returns, test_variance, model_specs):
    y_train = train_returns[asset].dropna()
    y_test = test_returns[asset].dropna()
    rv_test = test_variance[asset].dropna()

    forecast_var_dict = {}

    for name, spec in model_specs.items():
        print(f"\nRunning expanding forecast for {asset} - {name} ...")
        preds = []
```

```
        for i in range(len(y_test)):
            y_hist = pd.concat([y_train, y_test.iloc[:i]])

            model = arch_model(
                y_hist,
                vol=spec["vol"],
                p=spec["p"],
                q=spec["q"],
                o=spec["o"],
                mean="constant"
            )

            res = model.fit(disp="off")
            forecast = res.forecast(horizon=1)
            var_pred = forecast.variance.iloc[-1, 0]
            preds.append(var_pred)

        forecast_var_dict[name] = pd.Series(preds, index=y_test.index)

    rv_aligned = rv_test.loc[y_test.index]

    results_table = []
    for name, pred in forecast_var_dict.items():
        results_table.append({
            "Asset": asset,
            "Model": name,
            "MSE": mse_loss(rv_aligned, pred),
            "MAE": mae_loss(rv_aligned, pred),
            "QLIKE": qlike_loss(rv_aligned, pred)
        })

    return pd.DataFrame(results_table)

# Evaluate forecasts for all assets and compile results
assets = ["QQQ", "SPY", "DIA"]

all_results = []

for asset in assets:
    asset_result = evaluate_asset_forecast(
        asset,
        train_returns,
```

```
        test_returns,
        test_variance,
        model_specs
    )
    all_results.append(asset_result)

final_results_df = pd.concat(all_results, ignore_index=True)

print("\n===== Forecast Evaluation Results for All Assets =====")
print(final_results_df)
```

Appendix E: VaR forecasting and backtesting code

```
from arch import arch_model
from scipy.stats import norm, chi2
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Define training and testing periods
train_end = "2022-12-31"
test_start = "2023-01-01"

train_returns = log_returns_Rescaling.loc[:train_end]
test_returns = log_returns_Rescaling.loc[test_start:]

assets = ["QQQ", "SPY", "DIA"]

## Kupiec test function for backtesting VaR forecasts
def kupiec_test(violations, alpha):
    """
    Kupiec unconditional coverage test

    violations: boolean Series, True if VaR violation occurs
    alpha: expected violation probability (e.g. 0.05 or 0.01)
    """
    T = len(violations)
    x = violations.sum()

    # avoid log(0) problems in extreme cases
    if x == 0 or x == T:
```

```
        return np.nan, np.nan

    p_hat = x / T

    LR_uc = -2 * (
        (T - x) * np.log((1 - alpha) / (1 - p_hat)) +
        x * np.log(alpha / p_hat)
    )

    p_value = 1 - chi2.cdf(LR_uc, df=1)
    return LR_uc, p_value

# VaR forecasting and backtesting function for one asset
def var_forecast_backtest(asset, train_returns, test_returns):
    """
    For one asset:
    1. Fit GARCH(1,1) using expanding window
    2. Generate 1-step-ahead variance forecasts
    3. Convert variance forecasts to volatility forecasts
    4. Compute 95% and 99% VaR
    5. Count violations
    6. Run Kupiec test

    Returns:
        var_summary_df
        kupiec_df
        plot_df
    """
    y_train = train_returns[asset].dropna()
    y_test = test_returns[asset].dropna()

    # store variance forecasts
    var_preds = []

    for i in range(len(y_test)):
        # expanding window: training data + observed test data up to time i-1
        y_hist = pd.concat([y_train, y_test.iloc[:i]])

        model = arch_model(
            y_hist,
            vol="GARCH",
            p=1,
```

```
        q=1,
        o=0,
        mean="constant"
    )

    res = model.fit(disp="off")

    # one-step-ahead variance forecast
    forecast = res.forecast(horizon=1)
    var_pred = forecast.variance.iloc[-1, 0]
    var_preds.append(var_pred)

var_forecast = pd.Series(var_preds, index=y_test.index, name="Forecast_Variance")

# convert variance to volatility
vol_forecast = np.sqrt(var_forecast)

# use training sample mean as expected return estimate
mu_hat = y_train.mean()

# z-scores for left-tail VaR
z_95 = norm.ppf(0.05)   # about -1.645
z_99 = norm.ppf(0.01)   # about -2.326

# VaR forecasts
VaR_95 = mu_hat + z_95 * vol_forecast
VaR_99 = mu_hat + z_99 * vol_forecast

# actual returns in test period
actual_returns = y_test.loc[var_forecast.index]

# violations: actual return falls below VaR threshold
violations_95 = actual_returns < VaR_95
violations_99 = actual_returns < VaR_99

# counts and rates
n_test = len(actual_returns)

n_viol_95 = int(violations_95.sum())
n_viol_99 = int(violations_99.sum())

viol_rate_95 = n_viol_95 / n_test
```

```
    viol_rate_99 = n_viol_99 / n_test

    # VaR summary table
    var_summary_df = pd.DataFrame({
        "Asset": [asset, asset],
        "VaR Level": ["95%", "99%"],
        "Expected Violation Rate": [0.05, 0.01],
        "Actual Violation Rate": [viol_rate_95, viol_rate_99],
        "Number of Violations": [n_viol_95, n_viol_99],
        "Total Observations": [n_test, n_test]
    })

    # Kupiec backtest
    lr95, p95 = kupiec_test(violations_95, 0.05)
    lr99, p99 = kupiec_test(violations_99, 0.01)

    kupiec_df = pd.DataFrame({
        "Asset": [asset, asset],
        "VaR Level": ["95%", "99%"],
        "Kupiec LR Statistic": [lr95, lr99],
        "Kupiec p-value": [p95, p99]
    })

    # Data for plotting
    plot_df = pd.DataFrame({
        "Actual Return": actual_returns,
        "VaR 95%": VaR_95,
        "VaR 99%": VaR_99
    })

    return var_summary_df, kupiec_df, plot_df

# Run VaR forecasting and backtesting for all assets
all_var_summaries = []
all_kupiec_results = {}
all_plot_data = {}

for asset in assets:
    print(f"\nRunning VaR forecasting and backtesting for {asset}...")

    var_summary_df, kupiec_df, plot_df = var_forecast_backtest(
        asset,
```

```
        train_returns,
        test_returns
    )

    all_var_summaries.append(var_summary_df)
    all_kupiec_results[asset] = kupiec_df
    all_plot_data[asset] = plot_df

# combine all VaR summary rows
final_var_summary = pd.concat(all_var_summaries, ignore_index=True)

print("\n===== VaR Backtesting Summary for All Assets =====")
print(final_var_summary.to_string(index=False))

print("\n===== Kupiec Backtest Results =====")
for asset in assets:
    print(f"\n--- {asset} ---")
    print(all_kupiec_results[asset].to_string(index=False))


# Plot actual returns and VaR forecasts for each asset
for asset in assets:
    plot_df = all_plot_data[asset]

    plt.figure(figsize=(12, 6))
    plt.plot(plot_df.index, plot_df["Actual Return"], label="Actual Returns")
    plt.plot(plot_df.index, plot_df["VaR 95%"], label="95% VaR")
    plt.plot(plot_df.index, plot_df["VaR 99%"], label="99% VaR")
    plt.title(f"{asset}: Actual Returns and VaR Forecasts")
    plt.xlabel("Date")
    plt.ylabel("Return / VaR (%)")
    plt.legend()
    plt.show()
```